
Module 18 — Spring Modulith

Lesson Notes

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 The Modular Monolith	2
2.2 Modules in Spring Modulith	3
2.3 Events Between Modules	3
3. Step-by-Step Examples	3
3.1 Module Structure and <code>verify()</code>	4
3.2 Publishing an Event	5
3.3 Reacting: <code>@ApplicationModuleListener</code>	5
3.4 The Event Publication Registry	6
Externalization to Kafka	8
3.5 Module Tests: <code>@ApplicationModuleTest</code> and <code>Scenario</code>	8
3.6 Documentation	9
4. Common Mistakes and Best Practices	10
5. Summary	10
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Structure a Spring Boot application as a modular monolith, with one package per business module
- Check the module boundaries in a test with `ApplicationModules.verify()`
- Let modules talk through application events and `@ApplicationModuleListener`, not direct calls
- Explain what the event publication registry stores, and resubmit events whose listener failed
- Send selected events to Kafka with `@Externalized`
- Test one module at a time with `@ApplicationModuleTest` and `Scenario`
- Generate architecture documentation (C4 diagrams with PlantUML, module canvases)

Prerequisites: Module 06 (PostgreSQL, transactions), Module 11 (Kafka) · **Estimated time:** 4 hours · **Docker required**

2. Concepts

2.1 The Modular Monolith

A **modular monolith** is one deployable application whose code is divided into modules with clear boundaries. It keeps the simple operation of a monolith (one process, one database, one deployment) and gets much of the structure of microservices. A module that is well separated can later become its own service. That is much harder in a monolith where everything calls everything.

	Classic monolith	Modular monolith	Microservices
Deployment	one	one	many
Boundaries	by convention	checked by tests	enforced by the network
Calls between parts	any method	the module's API or events	HTTP, messaging
Consistency	one transaction	one transaction per module + events	eventual

2.2 Modules in Spring Modulith

Spring Modulith needs no configuration to find the modules:

- Every **direct sub-package** of the main application's package is a module: `order`, `inventory`, `catalog`, `notification`.
- The public types **in the module's package itself** are its API.
- **Sub-packages** of a module (`order.internal`) are internal, even when their classes are public for Java.

Rules that `verify()` checks: no cycles between modules, no access to another module's internal types, and only the dependencies a module allows (`allowedDependencies`).

2.3 Events Between Modules

When module A calls module B directly, A depends on B and must change when B changes. With an event, A only announces what happened (`OrderPlaced`), and any number of modules react. A knows none of them.

```

order —publishes→ OrderPlaced → inventory    (reserve stock)
                                ↳ notification (send confirmation)
                                ↳ Kafka topic   (other systems)
  
```

The listener runs **after the commit** of the publishing transaction, in its own transaction. So a failing listener does not roll back the order. But then who remembers that the stock was never reserved? That is the job of the **event publication registry**.

3. Step-by-Step Examples

With Docker running, start the application. PostgreSQL and Kafka start from the root `compose.yaml`:

```
./mvnw -pl modules/18-modulith/lesson spring-boot:run
```

The tour output:

```

=== 3.2-3.3 An order, and two modules that react to it ===
placed Order[id=1, customerId=tour, isbn=9780134685991, quantity=2, total=179.80]
inventory: 1000 → 998
mail: Thank you for order 1: 2 × 9780134685991, total 179.80
publication: inventory.StockReservations.on → COMPLETED
publication: notification.OrderConfirmation.on → COMPLETED
  
```

```

publication: EventExternalizerModuleListener.externalize → PROCESSING
=== 3.4 The warehouse is offline: the inventory listener fails ===
publication: inventory.StockReservations.on → FAILED
...
=== 3.4 Back online: resubmit the incomplete publications ===
publication: inventory.StockReservations.on → COMPLETED
...

```

3.1 Module Structure and verify()

The lesson application has four modules:

Module	API (module package)	Internal	Depends on
catalog	Book, CatalogService	catalog.internal.BookRepository	
order	Order, OrderPlaced, OrderService	order.internal.OrderRepository	
inventory	Inventory, Warehouse	StockReservations (package-private)	order (the event type)
notification	Mailbox	OrderConfirmation (package-private)	order (the event type)

A normal unit test checks the structure. It needs no Spring context and no Docker:

```

static final ApplicationModules MODULES =
    ↪ ApplicationModules.of(ModulithApplication.class);

@Test
void theModulesRespectTheirBoundaries() {
    MODULES.verify(); // no cycles, no access to another module's
    ↪ internal packages
}

```

If inventory used order.internal.OrderRepository, the test would fail with a message like Module 'inventory' depends on non-exposed type ...OrderRepository within module 'order'!

A module can also restrict its own dependencies. The notification module may only depend on order:

```
@ApplicationModule(allowedDependencies = "order")
```

i Note

Spring Modulith uses ArchUnit to read the compiled classes. Modulith 2.1.1 brings ArchUnit 1.4.2, which cannot read Java 27 class files (“No classes found”). The course’s build-parent therefore manages ArchUnit 1.5.0.

3.2 Publishing an Event

OrderPlaced is a record with only data: IDs, numbers, no entities. Listeners run later and in other transactions, and the registry stores the event as JSON:

```
@Externalized("bookstore.orders:#{customerId()}")    // also send to the Kafka topic,
↳ key = customer
public record OrderPlaced(long orderId, String customerId, String isbn, int quantity,
↳ BigDecimal total) {
}
```

The order service publishes it with Spring’s normal `ApplicationEventPublisher`:

```
@Transactional
public Order place(String customerId, String isbn, int quantity) {
    Book book = catalog.find(isbn).orElseThrow(() -> new UnknownBookException(isbn));
    BigDecimal total = book.price().multiply(BigDecimal.valueOf(quantity));
    Order order = repository.save(customerId, isbn, quantity, total);

    // stored in the registry in THIS transaction; delivered to the listeners after the
    ↳ commit
    events.publishEvent(new OrderPlaced(order.id(), customerId, isbn, quantity, total));
    return order;
}
```

The order module does not know who listens. `ModularityTest` checks that it depends only on `catalog`.

3.3 Reacting: `@ApplicationModuleListener`

```
@ApplicationModuleListener    // = @TransactionalEventListener + @Async +
↳ @Transactional(REQUIRES_NEW)
void on(OrderPlaced order) {
    inventory.reserve(order.isbn(), order.quantity());
}
```

```
warehouse.confirmReservation(order.isbn(), order.quantity()); // throws → rollback,
↪ publication FAILED
}
```

@ApplicationModuleListener combines three annotations:

Annotation	Effect
@TransactionalEventListener	runs only after the publishing transaction has committed
@Async	runs on another thread; the order request does not wait
@Transactional(propagation = REQUIRES_NEW)	the listener has its own transaction

The notification module has a second listener for the same event (OrderConfirmation). Neither listener knows the other.

3.4 The Event Publication Registry

With spring-modulith-starter-jdbc, every event gets **one row per listener** in the table event_publication, in the **same transaction** as the order. The table is created by Flyway (V2__create_event_publication.sql, copied from the Modulith 2.1 PostgreSQL schema).

Status	Meaning
PUBLISHED	the order was committed; the listener has not run yet
PROCESSING	the listener is running
COMPLETED	the listener finished successfully
FAILED	the listener threw an exception
RESUBMITTED	the publication was handed to the listener again

Configuration:

```
modulith:
  events:
    # the registry table is created by Flyway (V2); Modulith could also create it:
    # jdbc.schema-initialization.enabled: true
```

```

externalization:
  enabled: true                # send @Externalized events to Kafka (the default)
  completion-mode: update      # completed publications stay in the table with a
  ↪ completion date
kafka:
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    # no JSON value serializer: Modulith turns the event into JSON itself
    # (spring.modulith.events.kafka.enable-json, default true) – a JsonSerializer would
    ↪ encode it twice

```

The test switches the warehouse off, so the inventory listener fails. The order stays committed, and the registry remembers the failed publication. After the warehouse is back, the publication is resubmitted:

```

@Test
void aFailedListenerIsDeliveredAgainFromTheRegistry() {
    int before = inventory.available(JAVA_PUZZLERS);
    warehouse.goOffline();                // the inventory listener
    ↪ will throw

    Order order = orders.place("c-3", JAVA_PUZZLERS, 1);    // the order itself is
    ↪ committed

    await().atMost(Duration.ofSeconds(10))
        .untilAsserted(() ->
            ↪ assertThat(inventoryPublicationStatus(order.id())).isEqualTo("FAILED"));
    assertThat(inventory.available(JAVA_PUZZLERS)).isEqualTo(before);

    warehouse.goOnline();
    incompletePublications.resubmitIncompletePublications(
        publication -> publication.getEvent() instanceof OrderPlaced placed &&
    ↪ placed.orderId() == order.id());

    await().atMost(Duration.ofSeconds(10))
        .untilAsserted(() -> assert-
            ↪ That(inventoryPublicationStatus(order.id())).isEqualTo("COMPLETED"));
    assertThat(inventory.available(JAVA_PUZZLERS)).isEqualTo(before - 1);
}

```

In production, a scheduled job typically calls `resubmitIncompletePublicationsOlderThan(Duration)`, and `CompletedEventPublications.deletePublicationsOlderThan(...)` keeps the table small. With `spring.modulith.events.republish-outstanding-events-on-restart=true`, unfinished publications are also delivered again after a restart.

! Important

The registry guarantees **at least once**, not exactly once. After a crash, a listener may run twice for the same event. Listeners must be idempotent, or detect duplicates (compare module 11, the consumer side of the outbox).

Externalization to Kafka

`@Externalized("bookstore.orders::#{customerId()}")` on the event makes Modulith send it to the Kafka topic `bookstore.orders`, with the customer ID as key. This is also a listener with a registry row (`EventExternalizerModuleListener.externalize`): the registry works as an **outbox**. You do not write the outbox table and relay yourself, as in module 11. `ExternalizationTest` reads the topic with a plain Kafka consumer.

! Warning

Modulith turns the event into JSON itself (`spring.modulith.events.kafka.enable-json`, default `true`). If you also configure a JSON value-serializer, the event is encoded twice and arrives as a base64 string.

3.5 Module Tests: @ApplicationModuleTest and Scenario

`@ApplicationModuleTest` starts the context with the beans of **one** module only (the module of the test's package). Other modules are not there, and their beans can be mocked:

```
@ApplicationModuleTest                                // STANDALONE: the beans of this module
↳ only
@Import(TestcontainersConfiguration.class)
@TestPropertySource(properties = "bookstore.tour.enabled=false") // the tour needs all
↳ modules
class OrderModuleTest {

    @MockitoBean
    CatalogService catalog;                            // another module: replaced by a mock

    @Autowired
    OrderService orders;

    @Test
    void placingAnOrderPublishesOrderPlaced(Scenario scenario) {
        given(catalog.find("9780134685991"))
            .willReturn(Optional.of(new Book("9780134685991", "Effective Java", new
                ↳ BigDecimal("89.90"))));

        scenario.stimulate(() -> orders.place("c-1", "9780134685991", 2))
    }
}
```

```

        .andWaitForEventOfType(OrderPlaced.class)
        .matching(event -> event.customerId().equals("c-1"))
        .toArriveAndVerify(event ->
            ↪   assertThat(event.total()).isEqualByComparingTo("179.80"));
    }

```

Bootstrap mode	Started modules
STANDALONE (default)	only the module under test
DIRECT_DEPENDENCIES	plus the modules it depends on directly
ALL_DEPENDENCIES	plus all modules it depends on, transitively

Scenario describes a test as **stimulus** → **expected result**. The stimulus can be a method call (stimulate) or an event (publish). The result can be an event (andWaitForEventOfType) or a state change (andWaitForStateChange). Scenario waits for asynchronous listeners with Awaitility:

```

@Test
void anOrderReservesStock(Scenario scenario) {
    int before = inventory.available(ORDER_IN_ACTION);

    scenario.publish(new OrderPlaced(1000, "c-2", ORDER_IN_ACTION, 3, new
        ↪   BigDecimal("285.00")))
        .andWaitForStateChange(() -> inventory.available(ORDER_IN_ACTION), available
            ↪   -> available != before)
        .andVerify(available -> assertThat(available).isEqualTo(before - 3));
}

```

★ Tip

@ApplicationModuleTest also scans the application's root package. The Lesson-Tour there needs all modules, so the module tests turn it off with @TestPropertySource(properties = "bookstore.tour.enabled=false").

3.6 Documentation

```

@Test
void writeDocumentation() {
    new Docuener(MODULES).writeDocumentation();    // PlantUML (C4) diagrams + module
    ↪   canvases
}

```

```
assertThat(Path.of("target/spring-modulith-docs/components.puml")).exists();
assertThat(Path.of("target/spring-modulith-docs/module-order.puml")).exists();
}
```

After `./mvnw -pl modules/18-modulith/lesson test -Dtest=ModularityTest`, the folder `lesson/target/spring-modulith-docs` contains:

- `components.puml`: a C4 component diagram of all modules and their dependencies
- `module-<name>.puml`: one diagram per module
- `module-<name>.adoc`: a **module canvas** (API types, Spring beans, published and consumed events, properties)
- `all-docs.adoc`: everything in one document

Open the `.puml` files with a PlantUML plugin (IntelliJ IDEA, VS Code) or on plantuml.com. Because they are generated by a test, the documentation cannot drift away from the code.

4. Common Mistakes and Best Practices

!! Caution

Do not put entities or other mutable objects into events. The listener runs later, in another transaction, maybe after a restart from the JSON in the registry. Put in IDs and the values the listeners need.

- **Do:** design the module API on purpose: few public types in the module package, everything else in sub-packages.
- **Don't:** make a class public in an internal package and use it from another module “just this once”. `verify()` will fail, and that is the point.
- **Do:** run `verify()` (or `detectViolations()`) in every build. `ApplicationModules.of(...)` is cached per JVM, and after an `@ApplicationModuleTest` has verified it once, `verify()` does not check again. `detectViolations().throwIfPresent()` always checks.
- **Don't:** replace every method call with an event. Queries (“what does this book cost?”) stay calls to the other module’s API. Events are for “this has happened”.
- **Do:** make listeners idempotent and monitor FAILED publications.
- **Don't:** forget to clean up completed publications (`CompletedEventPublications`), or the table grows forever.

5. Summary

- Packages are modules: the module package is the API, and sub-packages are internal. `ApplicationModules.verify()` checks this in a unit test.

- Modules communicate with events. `@ApplicationModuleListener` runs after the commit, asynchronously, in its own transaction.
- The event publication registry stores one row per event and listener in the same transaction. Failed publications can be resubmitted: at-least-once delivery.
- `@Externalized` sends an event to Kafka, and the registry acts as the outbox.
- `@ApplicationModuleTest` starts one module, and Scenario tests events and state changes. Documenter generates C4 diagrams and module canvases.

Next step: [Exercises](#)

6. Further Reading

- [Spring Modulith Reference](#) · [Fundamentals](#) · [Verification](#)
- [Working with Application Events](#)
- [Integration Testing Application Modules](#)
- [Documenting Application Modules](#)
- [Martin Fowler — Monolith First](#)